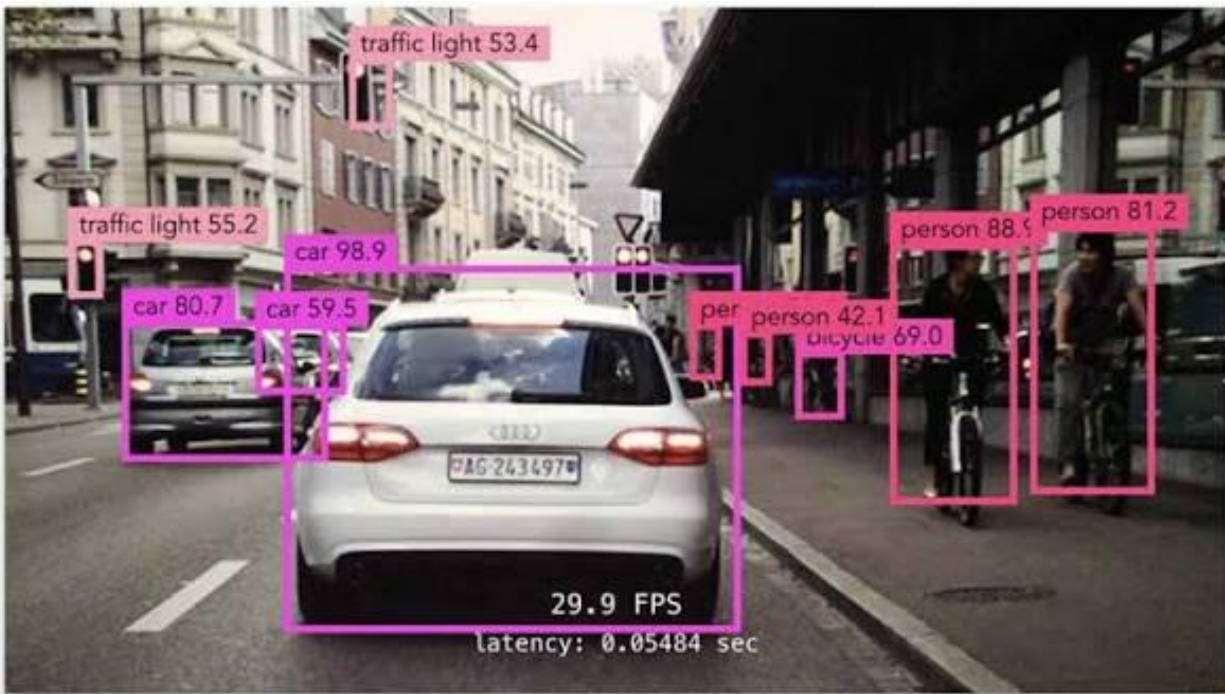


Lab 1 Report: Open Set Object Detection

with YOLOv11



1. An Overview of Open-Set Object Detection:

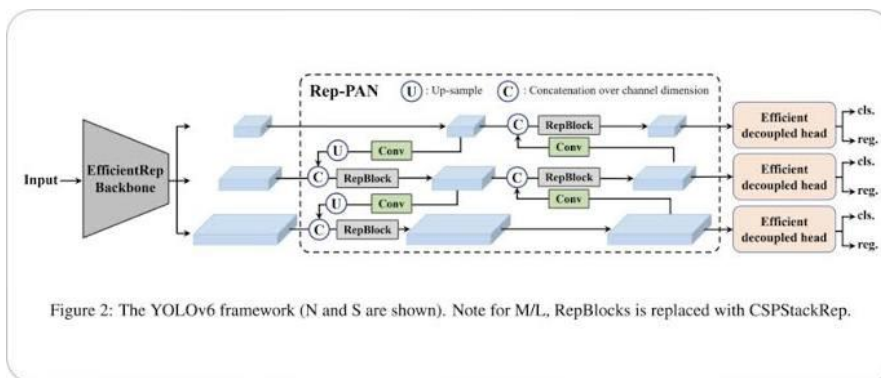
1.1 Open-Set object detection is a visual detection paradigm designed to handle images and environments where the model encounters multiple object categories that it wasn't necessarily exposed to during training. This differs from models like classical YOLO or R-CNNs in the sense that you are no longer just mapping the output over a set of known classes.

The OSOD model must fulfill two simultaneous goals:

1. Accurately predicting, localizing, and classifying objects belonging to the known categories.
2. Detecting and localizing objects of unknown categories instead of misclassifying them as known or swallowing them into the background

1.2 There are two core paradigms of Open-Set Object Detection: open-world object detection and open-vocabulary object detection. Open-world object detection seeks to detect unknown objects, flag them, and incrementally learn new classifications over time. However, modern OSOD frameworks can often leverage vision-language pre-training(for eg. image-text embeddings) instead of using fixed classification heads. This allows the user to query open-vocabulary categories dynamically at inference time. This methodology constitutes open-vocabulary object detection and allows the model to expand the vocabulary to concepts beyond the base training classes without complete re-training.

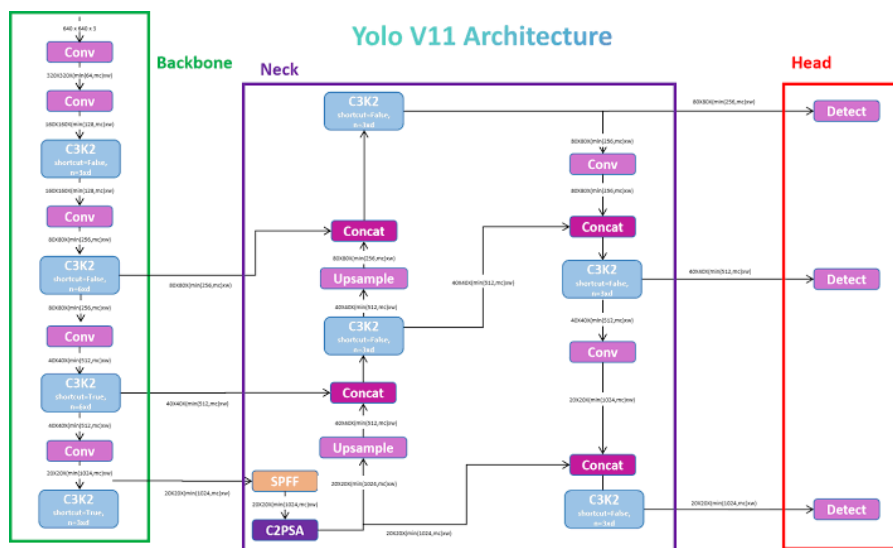
2. An Overview of the YOLOE Architecture and How It Was Modified to Perform Open-Set Object Detection:



2.1 The YOLOE architecture was designed to bring multi-prompt open-set detection and instance segmentation into the real-time YOLO(You Only Look Once) framework. It begins by extracting multi-scale visual features from the input image, then fuses high-level semantic features with low-level spatial detail across different scales to detect objects of varying sizes before using lightweight mask prototypes to perform simultaneous bounding box detection and instance segmentation.

2.2 YOLOE uses many strategies to function as an open-set detector, mainly by replacing the fixed linear classifier with dynamic region-prompt alignment strategies. It can also use a pre-computed large vocabulary space and specialized prompt embeddings in order to evaluate region proposals without large-model dependencies during prompt-free runs

3. How YOLOv11 Works:

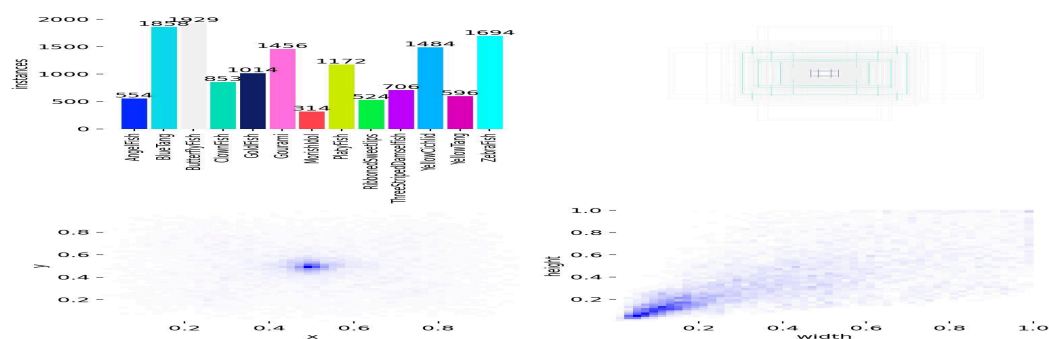


3.1 In the backbone, YOLOv11 replaces the standard C2f blocks with C3k2, which provides a lightweight implementation of Cross-Stage Partial structures with configurable kernel sizes. The Splitting feature maps into two pathways to capture multi-scale context faster with fewer parameters, making the entire process more efficient. Then, still in the backbone, YOLOv11 uses a C2PSA block, which integrates spatial attention directly into the feature collection stage. This essentially allows the model to focus selectively on high-importance objects without being distracted by background clutter.

The neck then fuses low-level spatial features with high-level semantic features using an updated Path Aggregation Network. Then the detection head creates bounding boxes and carries out object classification.

3.2 YOLOv11 achieves a higher mAP precision with up to 22% fewer parameters than YOLOv8m, which means it is faster and more efficient. Streamlined convolution blocks also allow for certain GPU optimizations, which add to the efficiency.

4. What I Learned From Fine-Tuning YOLOv11:



4.1 Fine-tuning allows us to take the base YOLOv11 model and train it to carry out specialized tasks. In my example, this will be recognizing pictures of certain kinds of fish. Since the pre-trained weights of the base YOLOv11 model are already fairly accurate, the initial learning rate is generally low in order to prevent overfitting. In addition to this, we employ a cosine learning rate scheduler, which decays this already minimal learning rate. You also have the option of choosing between AdamW or momentum SGD for the optimizer. AdamW generally converges faster, and SGD with momentum often yields a higher mAP on large datasets given the right learning rate scheduling. Essentially, the question is one of training efficiency versus potential mAP ceiling. Given that this model is already complex and fairly expensive to train, that is a tradeoff worth putting some thought into.

4.2 Operationally, the most important thing I found when using the YOLOv11 model, especially when using it on custom images, is making sure you are following the proper structure guidelines for your training set. This is a trivial thing and not exactly connected to the concept of YOLOE or Open-Set Object Detection, but it is important nevertheless. You must ensure you have the right directory structure, labels, and YAML file in the root of your custom data folder in order to carry out this fine-tuning. The other factor to keep in mind is the size and complexity of a model of this kind; even with a GPU, running a small 10-epoch training session takes a fair amount of time

4.3 The biggest thing to realize is that the baseline model is already very efficient, so your job during the fine-tuning is to make minor changes. This is why we start out with such a small learning rate and use a learning rate scheduler.